Union-Find (Disjoint Set Union)

> → **Key Idea**: Efficiently track connected components and detect cycles in undirected graphs using path compression and union by rank for nearly constant time operations.

> • **When to use**: Detect cycles in undirected graphs, find connected components, dynamic connectivity queries, Kruskal's MST algorithm

> ⊕ **Complexity**: Time O(α(n)) per operation with optimizations, Space O(n)

> ⚠ **Common Pitfall**: Forgetting path compression leads to O(n) per operation instead of O(α(n))

Overview

Visual Example

Union-Find Tree Evolution

```
Initial:      {1}   {2}   {3}   {4}   {5}

Union(1,2):   {1-2}   {3}   {4}   {5}

Union(3,4):   {1-2}   {3-4}   {5}

Union(1,3):   {1-2-3-4}   {5}
                   |
                 Root(1)
```

Path Compression Optimization

```
Before Compression:         After Compression (direct connection to root):
      1 (Root)                        1 (Root)
      |                              / | \
      2                             2  3  4
      |
      3
      |
      4

Find(4): Traverses 4->3->2->1, then connects 4 directly to 1.
```

Core Implementation

```
public class UnionFind {
    private int[] parent, rank;
    private int count; // Number of connected components

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        count = n;

        // Initialize: each node is its own parent
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            rank[i] = 0;
        }
    }

    /**
     * Find the root of the set containing element i
     * Uses path compression for optimization
     */
    public int find(int i) {
        if (parent[i] != i) {
            parent[i] = find(parent[i]); // Path compression
        }
        return parent[i];
    }

    /**
     * Union two sets containing elements i and j
     * Uses union by rank for balanced trees
     */
    public void union(int i, int j) {
        int rootI = find(i);
        int rootJ = find(j);

        if (rootI != rootJ) {
            // Union by rank: attach smaller tree under root of deeper tree
            if (rank[rootI] > rank[rootJ]) {
                parent[rootJ] = rootI;
            } else if (rank[rootI] < rank[rootJ]) {
                parent[rootI] = rootJ;
            } else {
                parent[rootJ] = rootI;
                rank[rootI]++; // Increase rank when trees have same height
            }
            count--; // Decrease number of connected components
        }
    }

    /**
     * Check if two elements are in the same set
```

```
     */
    public boolean connected(int i, int j) {
        return find(i) == find(j);
    }

    /**
     * Get number of disjoint sets
     */
    public int getCount() {
        return count;
    }
}
```

Key Optimizations

1. Path Compression

```
// Without path compression (slow)
public int findSlow(int i) {
    while (parent[i] != i) {
        i = parent[i];
    }
    return i;
}

// With path compression (fast)
public int find(int i) {
    if (parent[i] != i) {
        parent[i] = find(parent[i]); // Compress path
    }
    return parent[i];
}
```

2. Union by Rank

```
// Without union by rank (can create tall trees)
public void unionSlow(int i, int j) {
    int rootI = find(i);
    int rootJ = find(j);
    if (rootI != rootJ) {
        parent[rootJ] = rootI; // Always attach j to i
    }
}

// With union by rank (balanced trees)
public void union(int i, int j) {
    int rootI = find(i);
```

```
        int rootJ = find(j);
        if (rootI != rootJ) {
            if (rank[rootI] > rank[rootJ]) {
                parent[rootJ] = rootI;
            } else if (rank[rootI] < rank[rootJ]) {
                parent[rootI] = rootJ;
            } else {
                parent[rootJ] = rootI;
                rank[rootI]++;
            }
        }
    }
```

Common Problem Patterns

1. Number of Islands

Problem: Given a 2D binary grid which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are surrounded by water.

Example:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

```
    public int numIslands(char[][] grid) {
        if (grid == null || grid.length == 0) return 0;

        int rows = grid.length, cols = grid[0].length;
        UnionFind uf = new UnionFind(rows * cols + 1); // +1 for water dummy node

        int[][] directions = {{0,1}, {1,0}, {0,-1}, {-1,0}};
        int waterNode = rows * cols; // Dummy node for water

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (grid[i][j] == '0') {
                    // Connect water cells to dummy water node
                    uf.union(i * cols + j, waterNode);
                } else {
                    // Connect adjacent land cells
```

```
                for (int[] dir : directions) {
                    int newRow = i + dir[0];
                    int newCol = j + dir[1];

                    if (newRow >= 0 && newRow < rows &&
                        newCol >= 0 && newCol < cols &&
                        grid[newRow][newCol] == '1') {
                        uf.union(i * cols + j, newRow * cols + newCol);
                    }
                }
            }
        }
    }

    return uf.getCount() - 1; // Subtract dummy water component
}
```

2. Friend Circles (Connected Components)

Problem: There are n people in a social network. You are given an n x n matrix isConnected where isConnected[i][j] = 1 if person i and person j are directly connected, and isConnected[i][j] = 0 otherwise. Return the total number of friend circles (connected components).

Example:

```
Input: isConnected = [[1,1,0],[1,1,0],[0,0,1]]
Output: 2
Explanation: People 0 and 1 are friends, person 2 is alone. So there are 2 friend circle
```

```
public int findCircleNum(int[][] isConnected) {
    int n = isConnected.length;
    UnionFind uf = new UnionFind(n);

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (isConnected[i][j] == 1) {
                uf.union(i, j);
            }
        }
    }

    return uf.getCount();
}
```

3. Graph Valid Tree (Cycle Detection)

Problem: You have a graph of n nodes labeled from 0 to n - 1. You are given an integer n and a list of edges where edges[i] = [ai, bi] indicates that there is an undirected edge between nodes ai and bi in the graph. Return true if the edges of the given graph make up a valid tree, and false otherwise.

Example:

```
Input: n = 5, edges = [[0,1],[0,2],[0,3],[1,4]]
Output: true
Explanation: The graph forms a valid tree.

Input: n = 5, edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]
Output: false
Explanation: The graph contains a cycle.
```

```java
public boolean validTree(int n, int[][] edges) {
    // A valid tree has exactly n-1 edges and no cycles
    if (edges.length != n - 1) return false;

    UnionFind uf = new UnionFind(n);

    for (int[] edge : edges) {
        // If nodes are already connected, adding this edge creates a cycle
        if (uf.connected(edge[0], edge[1])) {
            return false;
        }
        uf.union(edge[0], edge[1]);
    }

    return uf.getCount() == 1; // Should be exactly one component
}
```

4. Accounts Merge

Problem: Given a list of accounts where each element accounts[i] is a list of strings, where the first element accounts[i][0] is a name, and the rest of the elements are emails representing emails of the account. Merge accounts that belong to the same person. Two accounts belong to the same person if there is some common email to both accounts.

Example:

```
Input: accounts = [["John","johnsmith@mail.com","john_newyork@mail.com"],
                   ["John","johnsmith@mail.com","john00@mail.com"],
                   ["Mary","mary@mail.com"],
                   ["John","johnnybravo@mail.com"]]
Output: [["John","john00@mail.com","john_newyork@mail.com","johnsmith@mail.com"],
         ["Mary","mary@mail.com"],
```

```
        ["John","johnnybravo@mail.com"]]
Explanation: The first and second John's are the same person as they have the common ema
```

```java
public List<List<String>> accountsMerge(List<List<String>> accounts) {
    Map<String, Integer> emailToIndex = new HashMap<>();
    Map<String, String> emailToName = new HashMap<>();

    int index = 0;

    // Map emails to indices and names
    for (List<String> account : accounts) {
        String name = account.get(0);
        for (int i = 1; i < account.size(); i++) {
            String email = account.get(i);
            if (!emailToIndex.containsKey(email)) {
                emailToIndex.put(email, index++);
            }
            emailToName.put(email, name);
        }
    }

    UnionFind uf = new UnionFind(index);

    // Union emails from same account
    for (List<String> account : accounts) {
        String firstEmail = account.get(1);
        for (int i = 2; i < account.size(); i++) {
            uf.union(emailToIndex.get(firstEmail), emailToIndex.get(account.get(i)))
        }
    }

    // Group emails by root
    Map<Integer, List<String>> rootToEmails = new HashMap<>();
    for (String email : emailToIndex.keySet()) {
        int root = uf.find(emailToIndex.get(email));
        rootToEmails.computeIfAbsent(root, k -> new ArrayList<>()).add(email);
    }

    // Build result
    List<List<String>> result = new ArrayList<>();
    for (List<String> emails : rootToEmails.values()) {
        Collections.sort(emails);
        List<String> account = new ArrayList<>();
        account.add(emailToName.get(emails.get(0))); // Add name
        account.addAll(emails); // Add sorted emails
        result.add(account);
    }

    return result;
}
```

Advanced Variations

Union-Find with Size Tracking

```
public class UnionFindWithSize {
    private int[] parent, size;
    private int count;

    public UnionFindWithSize(int n) {
        parent = new int[n];
        size = new int[n];
        count = n;

        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1; // Each component initially has size 1
        }
    }

    public int find(int i) {
        if (parent[i] != i) {
            parent[i] = find(parent[i]);
        }
        return parent[i];
    }

    public void union(int i, int j) {
        int rootI = find(i);
        int rootJ = find(j);

        if (rootI != rootJ) {
            // Union by size: attach smaller component to larger
            if (size[rootI] < size[rootJ]) {
                parent[rootI] = rootJ;
                size[rootJ] += size[rootI];
            } else {
                parent[rootJ] = rootI;
                size[rootI] += size[rootJ];
            }
            count--;
        }
    }

    public int getComponentSize(int i) {
        return size[find(i)];
    }
}
```

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Operation	Time Complexity	Space Complexity
Constructor	O(n)	O(n)
Find (with path compression)	O(α(n))*	O(1)
Union (with union by rank)	O(α(n))*	O(1)
Connected	O(α(n))*	O(1)

*α(n) is the inverse Ackermann function, which grows extremely slowly and is effectively constant for all practical purposes.

← Previous: Main Guide | Back to Main Guide | Next: Binary Search →